Time-Mapped Spectral Method for Singularity Formation in Inviscid Burgers' Equation

Stanley Mottram

I certify that this project report has been written by me, is a record of work carried out by me, and is essentially different from work undertaken for any other purpose or assessment.

Abstract

This project investigates the formation of singularities in partial differential equations through the inviscid Burgers' equation. The linear advection equation is first considered as a baseline problem where analytical solutions are known and no singularities form, allowing the numerical methods to be validated. The focus is then shifted to the non-linear case, where the dependence of the velocity on the solution leads to shock formation in finite time, characterised by the blow-up of the spatial derivative.

Numerical solutions are obtained using a first-order upwind scheme, which captures the general behaviour of the solution but loses accuracy as the shock time is approached. To better analyse behaviour near the singularity, a time-mapping technique is introduced, reformulating the problem so that the singularity occurs at infinite mapped time. While this improves the ability to study the solution close to blow-up, significant numerical errors remain. To address this, a pseudo-spectral method with a Fourier basis is implemented, providing much higher spatial accuracy. This leads to a substantial reduction in error and allows the solution to be resolved much closer to singularity formation, demonstrating the importance of high-order methods for studying non-linear PDE behaviour.

Contents

1 Introduction **1**

2 Linear Advection Equation **3**

2.1 Analytical Solution of the Standard Form 3

2.2 Numerically Solving Linear Advection 4

2.3 Numerical Results 5

3 Inviscid Burgers' Equation **8**

3.1 Analytical Solution of the Standard Form 8

3.2 Numerically Solving Inviscid Burgers' Equation 8

3.3 Numerical Results 10

4 Alternative Mapping for Time **14**

4.1 Defining the Mapping 14

4.2 Mapping the Inviscid Burgers' Equation 15

4.3 Numerical Results 15

5 Spectral Method with Fourier Basis **18**

5.1 Defining the Spectral Method 18

5.2 Implementing the Method 18

5.3 Numerical Results 20

6 Conclusion **24**

A Python Code for the Numerical Methods Used **28**

1 Introduction

The study of singularity formation in partial differential equations is of fundamental importance in both mathematics and physics, Jost (2002). Singularities represent the breakdown of classical solutions, where quantities such as gradients become unbounded in finite time, signalling the limits of standard analytical and numerical methods. Understanding when and how such singularities form is crucial for developing more robust mathematical models and for improving numerical methods that remain accurate near these extreme events.

A particularly important motivation comes from fluid dynamics, where the question of singularity formation remains one of the most significant open problems. The Navier–Stokes existence and smoothness problem, one of the Clay Mathematics Institute Millennium Prize Problems, Fefferman (2000), concerns whether smooth solutions to the three-dimensional incompressible Navier–Stokes equations can develop singularities in finite time. Despite extensive research, this question remains unresolved, Tao (2007). Simpler model equations, such as the inviscid Burgers' equation, provide a valuable testing ground for developing analytical and numerical techniques to study blow-up behaviour in a controlled setting, Tadmor (1989).

Furthermore, accurately capturing the behaviour of solutions near singularities presents a significant numerical challenge. Standard low-order methods often suffer from instability or loss of accuracy as gradients steepen. This motivates the use of high-accuracy approaches, that are better suited to resolving smooth solutions up to the onset of singularity.

This project aims to study the formation of singularities in partial differential equations using different numerical methods, with a particular focus on the inviscid Burgers' equation, which is defined as

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = 0, \tag{1.1}$$

where u, a function of time t and space x, is the speed of an inviscid fluid moving in the x-direction.

Burgers' equation provides one of the simplest non-linear PDEs in which finite-time singularities, in the form of shock waves, can occur, making it an ideal model problem for studying the breakdown of classical solutions, Burgers (1948). The

inviscid Burgers' equation arises as a simplified model of non-linear advection in fluid flow, obtained by neglecting viscosity and pressure effects. It can be viewed as a one-dimensional representation of the Navier–Stokes equations, retaining the non-linear term but omitting the viscous term.

To build up to this, the linear advection equation is first considered as a simpler case where singularities do not occur and solutions remain smooth for all time. Both analytical and numerical solutions are examined, providing validation before moving to the non-linear case.

The inviscid Burgers' equation is then studied, where the transport velocity depends on the solution itself. In this case, characteristic curves can intersect, leading to a blow-up of the spatial derivative in finite time. Numerical methods using finite differences are used to approximate the solution up to just before the shock time, and the behaviour of the infinity norm of the gradient is analysed as the singularity is approached.

To analyse the singularity with more detail, an alternative time-mapping technique is implemented to reformulate the problem in a way that removes the finite-time singularity, allowing the dynamics near shock formation to be studied in greater detail, closer to the singularity time. This mapping proves useful, but the errors found increase without bound far earlier than expected when compared to Bustamante (2011), the paper the mapping is based on. To fix this a higher-order numerical method is used to improve the accuracy of the results.

The numerical method used was the pseudo-spectral method with a Fourier basis, providing a significant improvement over the finite difference approaches considered initially. By representing the solution in terms of Fourier modes, spatial derivatives are computed with spectral accuracy, Trefethen (2000), allowing smooth solutions to be resolved with far fewer grid points compared with other numerical methods. The non-linear term is evaluated using a pseudo-spectral formulation, ensuring computational efficiency while maintaining high accuracy. As a result, the numerical solution remains stable and highly accurate up to times close to shock formation, where gradients become steep. Compared to earlier low-order methods, the spectral approach produces substantially reduced errors and captures the evolution of the solution much more accurately, particularly in the pre-singularity area.

2 Linear Advection Equation

2.1 Analytical Solution of the Standard Form

The linear advection equation provides a basic starting step for studying the inviscid Burgers' equation. The equation is

$$u_t + vu_x = 0 \quad \text{where} \quad v = v(x,t). \tag{2.1}$$

The velocity v controls the speed of the "transport" of the solution, and the subscripts t or x of u denote the derivative of u with respect to that variable. The initial condition is $u(x,0) = f(x)$ and we consider a periodic boundary $u(a,t) = u(b,t)$ and domain $x \in [a,b)$.

Equation (2.1) can be solved using the method of characteristics, setting

$$\begin{aligned} \dot{t} &= \frac{dt}{ds} = 1, \\ \dot{x} &= \frac{dx}{ds} = v, \\ \dot{u} &= \frac{du}{ds} = 0. \end{aligned} \tag{2.2}$$

Taking the ratio of $\dot{t}$ and $\dot{x}$, we obtain

$$\frac{dx}{dt} = v \implies x = vt + C, \quad \frac{du}{dt} = 0 \implies u = c, \tag{2.3}$$

which defines the characteristic curves, shown in figure 2.1. Since $u = c$ and $x - vt = C$, are constant in time, we can relate them giving,

$$u = f(x - vt), \tag{2.4}$$

where the function f is defined in the initial conditions.

To ensure the existence of a solution, the initial condition $u(x,0) = f(x)$ must be sufficiently smooth and periodic on the domain $[a,b)$. Under these assumptions, the characteristic curves are well-defined and do not intersect provided the advection velocity field v remains smooth. In the case where v is constant,

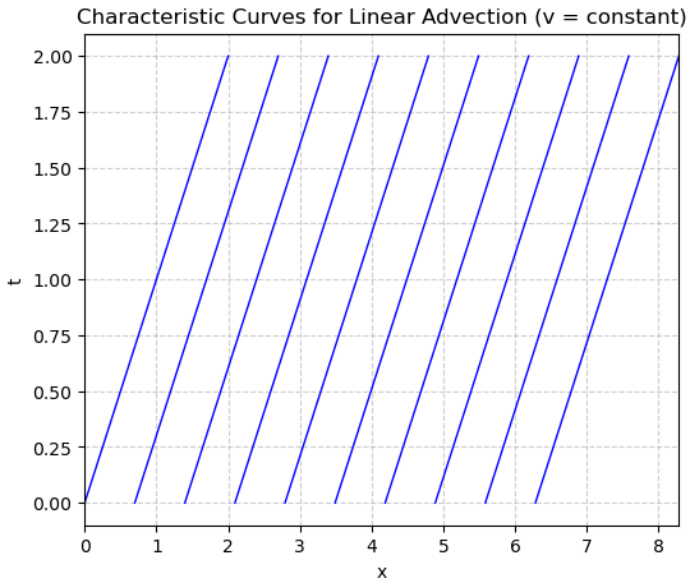


Figure 2.1: *The characteristic curves for the linear advection equation with constant velocity ($v = 1$) are straight, parallel lines. Since these curves do not intersect, the solution remains smooth for all time and no singularities form.*

the characteristics, given by (2.3), are straight, parallel lines, and the solution represents a simple translation of the initial profile without change in shape. As a result, the solution remains smooth for all time and no singularities form. More generally, if v is smooth and independent of u, the equation remains linear and the solution preserves the regularity of the initial data.

2.2 Numerically Solving Linear Advection

To numerically solve equation (2.1), a first-order upwind scheme with forward Euler in time method was used (Lucas 2025). A space-time grid is defined with N spatial points where the jth step in space and the nth step in time are defined as $x_j = a + j\Delta x$ and $t^n = n\Delta t$ respectively. With $\Delta x = (b - a)/N$ and Δt the size of the time step (using the Courant–Friedrichs–Lewy condition (CFL) defined at the end of this section). The numerical approximation of the solution is then denoted as

$$u_j^n \approx u(x_j, t^n). \tag{2.5}$$

Time discretisation uses the forward Euler method and is defined as

$$\frac{\partial u}{\partial t} \approx \frac{u_j^{n+1} - u_j^n}{\Delta t} \implies u_j^{n+1} \approx u_j^n - \Delta t v_j^n \left(\frac{\partial u}{\partial x}\right)_j^n,$$

with v_j^n being the value of v at the jth step in space and the nth step in time.

The spatial derivative is approximated using the upwind finite difference scheme. If the value of v_j^n is positive, then the backward difference is used and if v_j^n is negative, then the forward difference is used, so that

$$\left(\frac{\partial u}{\partial x}\right)_j^n \approx \begin{cases} \dfrac{u_j^n - u_{j-1}^n}{\Delta x}, & \text{if } v_j^n > 0, \\ \dfrac{u_{j+1}^n - u_j^n}{\Delta x}, & \text{if } v_j^n < 0. \end{cases} \tag{2.6}$$

Therefore, the final formula for the scheme is

$$u_j^{n+1} \approx u_j^n - \frac{\Delta t}{\Delta x} v_j^n \begin{cases} u_j^n - u_{j-1}^n, & v_j^n > 0, \\ u_{j+1}^n - u_j^n, & v_j^n < 0. \end{cases} \tag{2.7}$$

At points where $v_j^n = 0$, there is no preferred upwind direction, either one can be used.

For this scheme to be stable the Courant–Friedrichs–Lewy condition (CFL) must be satisfied, first discussed in Courant, Friedrichs, and Lewy (1928),

$$c = \max_{t,x} |v(x,t)| \ \frac{\Delta t}{\Delta x} \leq 1. \tag{2.8}$$

If this condition is violated, then the behaviour of the equation cannot be tracked by the grid of the numerical approximation and the results become numerically unstable. For that reason c is forced to be 0.9 and Δt is defined so that this condition remains satisfied.

The upwind scheme is used because it is a relatively stable method (provided that the CFL condition is met) and also requires less computational power compared to higher order methods. Although this upwind scheme is only first-order accurate (Lucas 2025), it is very robust and does not create oscillations. Defining oscillations as wiggles or overshoots in the numerical solution, particularly near steep gradients or discontinuities. These are commonly observed in higher-order schemes, such as spectral methods, Trefethen (2000) . This makes it a reliable choice for transport problems like linear advection where preserving the shape and direction of the solution is important.

2.3 Numerical Results

Two examples were tested, each with a resolution of 50 grid points in x. The initial conditions used to test this scheme were

1.
$$f(x) = \sin(x), \quad 0 \leq x \leq 2\pi, \quad 0 \leq t \leq 2, \quad v = 1,$$

2.
$$f(x) = 1 + \cos(x), \quad -2\pi \leq x \leq 2\pi, \quad 0 \leq t \leq 4, \quad v = 1.$$

The periodic boundary conditions are satisfied since $sin(0) = sin(2\pi) = 0$ and $1 + \cos(-2\pi) = 1 + \cos(2\pi) = 2$.

In both examples in figure 2.2, the solution profile translates uniformly in the positive x-direction as time increases, consistent with the exact solution of the linear advection equation. The overall shape in each example is largely preserved, indicating that the numerical scheme captures the advective transport accurately over the simulated time interval.

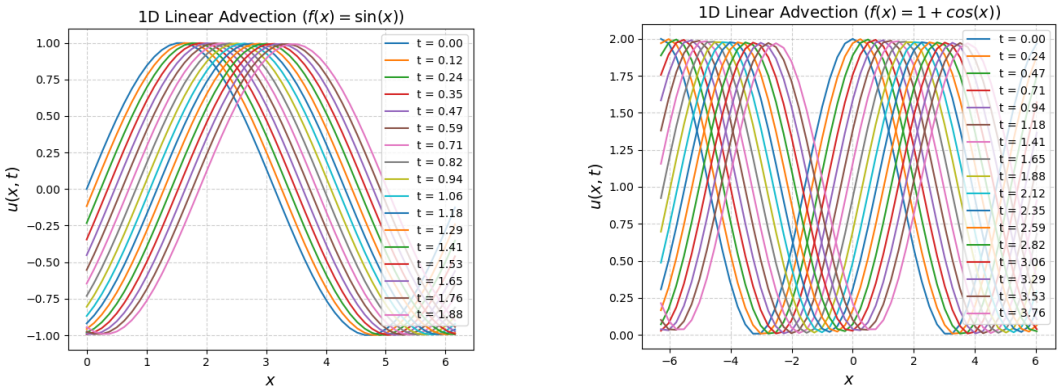


Figure 2.2: *Two results from implementing the upwind scheme using Python. Ten snapshots of time have been plotted, equally spaced out in their ranges for t.* ***Left panel:*** $f(x) = \sin(x)$ *initial condition was used with* $v = 1$*, exact solution of* $u = \sin(x - t)$. ***Right panel:*** $f(x) = 1 + \cos(x)$ *initial condition was used with* $v = 1$*, exact solution of* $u = 1 + \cos(x - t)$.

However, small numerical errors are visible near the extrema of both waves, where wave height decreases with each time step, particularly at later times. The error increases linearly through time when looking at figure 2.3. This is because the upwind scheme is first-order accurate, so the numerical error scales proportionally with the grid spacing, meaning a linear decrease in error as the spatial step size decreases would be expected.

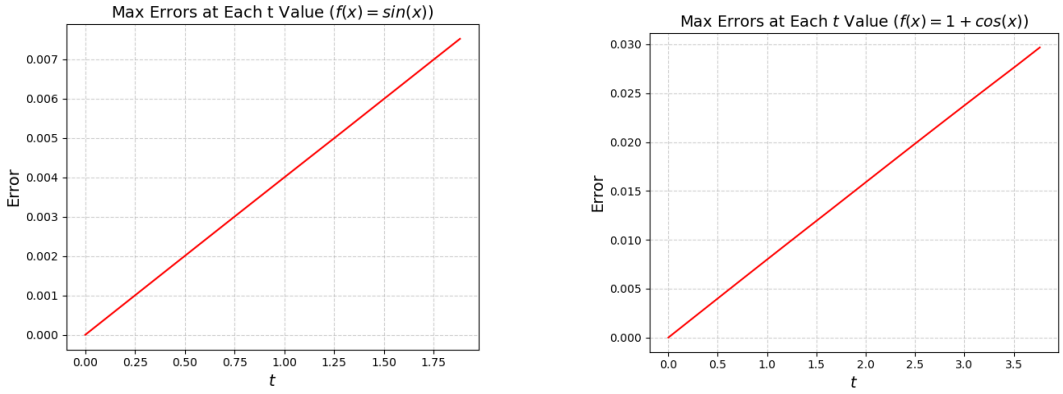


Figure 2.3: *The max errors at each t value graphed, the max error is defined as the point at each t as* $\max_x |u_{approx} - u_{exact}|$. *Both examples show linearly increasing errors as time advances.*

3 Inviscid Burgers' Equation

3.1 Analytical Solution of the Standard Form

The equation for the inviscid Burgers' equation is

$$u_t + uu_x = 0 \quad \text{where now} \quad v = u(x,t). \tag{3.1}$$

The initial condition is still $u(x,0) = f(x)$, still with a periodic boundary $u(a,t) = u(b,t)$ and domain $x \in [a,b)$. This can be solved similarly to the linear advection equation using the method of characteristics

$$\begin{aligned} \dot{t} &= \frac{dt}{ds} = 1, \\ \dot{x} &= \frac{dx}{ds} = u, \\ \dot{u} &= \frac{du}{ds} = 0. \end{aligned}$$

Using the characteristics equations, and combining $\dot{t}$ and $\dot{x}$

$$\frac{dx}{dt} = u \implies x = ut + C, \quad \frac{du}{dt} = 0 \implies u = c.$$

Since $u = c$ and $x - ut = C$, we can again relate the constants,

$$u = f(x - ut), \tag{3.2}$$

for the function f defined in the initial condition. The characteristics can be seen in figure (3.1), the horizontal dashed line is when the singularity first forms.

3.2 Numerically Solving Inviscid Burgers' Equation

Only small changes have to be made to solve the inviscid Burgers' equation numerically in Python. The final formula is

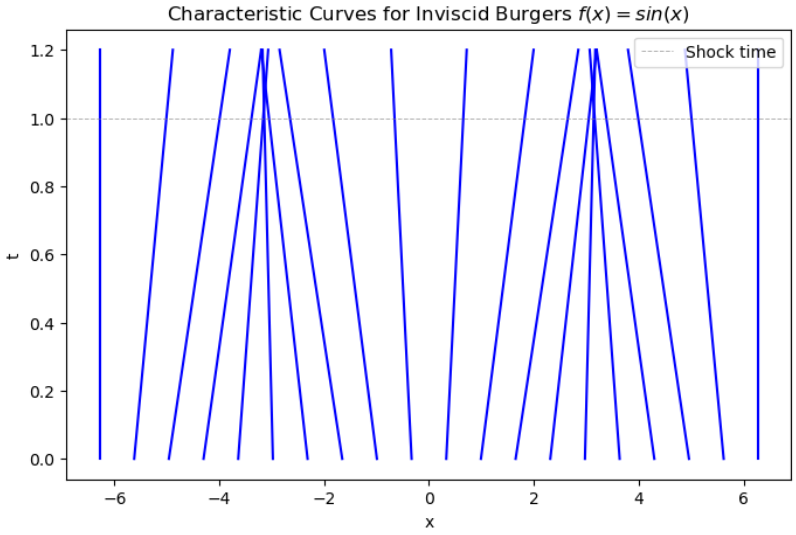


Figure 3.1: *The characteristic curves for the inviscid Burgers' equation converge to meet. At the earliest point these curves intersect a singularity has formed. Here it can be seen to be at* $t = 1$.

$$u_j^{n+1} \approx u_j^n - \frac{\Delta t}{\Delta x} u_j^n \begin{cases} u_j^n - u_{j-1}^n, & u_j^n > 0, \\ u_{j+1}^n - u_j^n, & u_j^n < 0, \end{cases} \tag{3.3}$$

with the only difference being the u instead of v. For this scheme to remain be stable the Courant–Friedrichs–Lewy condition (CFL) still must be satisfied

$$c = \max_{t,x} |f(t,x)| \ \frac{\Delta t}{\Delta x} \leq 1. \tag{3.4}$$

To make sure this is still true, c is again forced to be 0.9 and Δt is defined so that this condition remains satisfied.

Extra care must be taken when solving the equation as singularities (or shocks) now appear. These cannot be captured, as the infinity norm of the slope tends to infinity $\|\partial u/\partial x\|_{L^\infty} \to \infty$. The shock time must be found so that the numerical method only models the equation up to just before that time.

Before this time, the solution can be described by characteristics, which satisfy $x = x_0 + f(x_0)t$ and along which u is constant. If the initial data has regions where $f'(x_0) < 0$, faster characteristics catch up with slower ones, causing them to intersect; the "shock time" is precisely the first time such an intersection occurs. It is given by

$$t_s = -\frac{1}{\min_x f'(x)}, \tag{3.5}$$

where the minimum is taken over all points where $f'(x) < 0$. Equation (3.5) is obtained from the method of characteristics by finding when characteristic curves intersect, which occurs when $1 + f'(x_0)t = 0$, and taking the earliest such time. At $t = t_s$, the solution gradient becomes infinite, the classical solution breaks down.

An analysis on $\|\partial u/\partial x\|_{L^\infty}$ was conducted to see what happens to the gradient of the solution as t approaches the shock time. After finding the values of the gradient $\partial u/\partial x$ in Python, to find an accurate value for the infinity norm, polynomial interpolation was conducted with 3 points by approximating the true maximum between grid points, which may otherwise be missed due to the discrete resolution of the numerical solution. The points used were the grid point with the highest gradient and the two points either side, the highest point on the polynomial was taken as the infinity norm. These values are then plotted and curves were fit based on the assumption that it had the shape of

$$\left\| \frac{\partial u}{\partial x} \right\|_{L^\infty} = \frac{c}{(T^\star - t)^k}. \tag{3.6}$$

The infinity norm of the gradient is expected to diverge as the solution approaches shock time. This divergence typically follows a power law of this form, where $T^\star$ is the approximated shock time, different from the known shock time t_s found using equation (3.5), c and k are constants. The parameters (c,k) are estimated using non-linear least squares fitting, which minimises the squared error between the model and the data. A logarithmic transformation can also be applied to linearise the power-law behaviour, improving numerical stability and allowing the blow-up rate to be estimated more reliably.

3.3 Numerical Results

The example tested using this scheme is

$$f(x) = \sin(x), \quad 0 \le x \le 2\pi, \quad 0 \le t < t_s. \tag{3.7}$$

The range for t must stop before the shock time (the solutions breakdown after that and stop being valid). Multiple resolutions were tested, nineteen uniform resolutions, starting at 100 and ending at 1000 to provide a good range of resolutions without excessive computational power required. The errors were found by substituting the data approximated $u_{approx}(x, t)$ back into the equation $u = f(x - ut)$, then subtracting these from one another.

$$u = f(x - u_{approx}t), \quad error = |u - u_{approx}|.$$

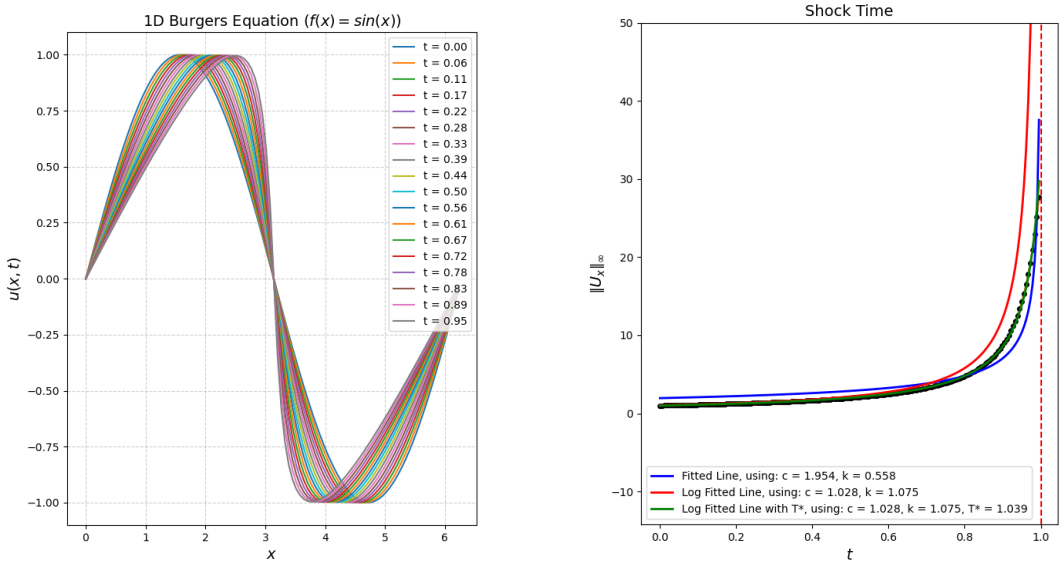


Figure 3.2: *Results from numerically finding the solution to the inviscid Burgers' equation using the modified upwind scheme at a resolution of* 1000. ***Left panel:*** *Numerical solution found with the* $f(x) = \sin(x)$ *initial condition. Multiple snapshots of time have been plotted, equally spaced out in the range for* t. ***Right panel:*** *The plotted infinity norm values (black dots) and three fitted curves. The red and green fits were found by fitting* c, k *to the log of equation* (3.6). *The green line also fitted* $T^\star$ *for the shock time* t_s *and didn't use equation* (3.5).

In figure 3.2 the plotted results on the left panel clearly shows a shock forming as $t \to t_s$ which in this case is found to be approximately 1, this was found by using equation (3.5) and estimating the minimum gradient of the initial condition f. However, in this case, t_s can be found analytically to be exactly 1 by finding the minimum of $f'(x) = \cos(x)$ to be -1 and substituting it into equation (3.5). It is much harder to see any visible errors at this resolution. The left panel in figure 3.3 shows that the largest error is approximately 0.0077 at $t \approx 1$, near the shock time t_s. In the right panel of figure 3.2 the blue line is fitted using the approximated shock time and without any logarithmic transformation. The red line does use a logarithmic transformation, this results in a better fit at earlier t values compared to the blue line. The green line does not use the approximated shock time and instead estimates its value, it can clearly be seen to be the best fit for the plotted results. The best fitted curve found by the green fit is

$$\left\| \frac{\partial u}{\partial x} \right\|_{L^\infty} \approx \frac{1.028}{(1.039 - t)^{1.075}}. \tag{3.8}$$

Equation (3.8) provides a extremely good fit for the results found for the infinity norm. In this case the equation can be found analytically by solving equation (3.1) to get

$$\left\| \frac{\partial u}{\partial x} \right\|_{L^\infty} = \frac{1}{1 - t}. \tag{3.9}$$

So in theory, c, k and $T^\star$ should all equal 1.

The right panel in figure 3.3 shows that increasing the resolution drastically increases the accuracy of the results. To make this a fair comparison all parameters were kept the same except the resolution and the error at $t = 0.5$ was taken so ensure the results were still stable and hadn't tended to infinity (which clearly happens near t_s, shown in the left panel of figure 3.3).

The left panel in figure 3.3 shows the size of the errors for each t value using the highest resolution of $N = 1000$. The closer to the shock time, the errors appear to get exponentially larger. The right panel shows the errors can be seen to drastically decrease as the resolution increases, showing that the method is more reliable with more grid points. In fact, the errors appear to be decreasing in proportion to the number of grid points ($error \propto \Delta x$), the error roughly halves as the resolution doubles. This is first-order convergence and is expected since the method used is first-order. The errors were taken at $t = 0.5$ because closer to the shock time, the results become less and less reliable due to the numerical method only being first-order. If the errors are taken at $t = 0.5$, it ensures numerical stability for analysing how increasing resolutions affect accuracy.

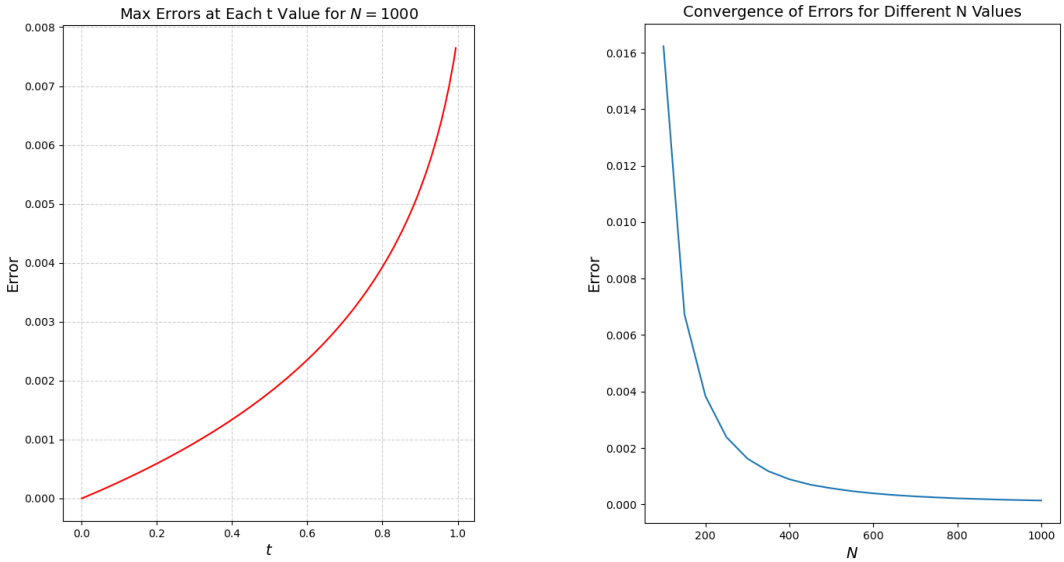


Figure 3.3: *Error analysis from numerically finding the solution to the inviscid Burgers' equation using the upwind scheme.* ***Left panel:*** *The absolute size of the errors at all t values for a resolution of* 1000. ***Right panel:*** *The errors found at $t = 0.5$ for each resolution N.*

4 Alternative Mapping for Time

4.1 Defining the Mapping

The mapping used in this section follows the construction introduced in Bustamante (2011), where the inviscid Burgers equation is reformulated through a non-linear time rescaling. In the original time variable, quantities such as the gradient can grow extremely rapidly near t_s, making them difficult to resolve numerically. By introducing a mapped time variable, this rapid growth is effectively slowed down, allowing finer resolution of the solution's behaviour close to the critical time. This will make it easier to analyse the formation of singularities.

The rescaled time variable τ and v are defined by

$$\tau(t) = \int_0^t \|u_x(\cdot, t')\|_\infty \, dt', \qquad v(x, \tau) = \frac{u(x, t)}{\|u_x(\cdot, t)\|_\infty}. \tag{4.1}$$

This transformation maps the interval $t \in [0, t_s)$ onto $\tau \in [0, \infty)$, pushing the singularity to infinite mapped time. In addition to the time rescaling, the velocity field is normalised using the supremum norm of its spatial derivative. This leads naturally to a mapping between the energies of the original and rescaled systems. The energy of the original inviscid Burgers equation is defined as

$$E \equiv \frac{1}{2} \int_0^{2\pi} |u(x, t)|^2 \, dx, \tag{4.2}$$

which is conserved for all $t < t_s$. Correspondingly, the mapped energy is defined by

$$E_{\text{map}}(\tau) \equiv \frac{1}{2} \int_0^{2\pi} |v(x, \tau)|^2 \, dx. \tag{4.3}$$

The relationship between E and $E_{\text{map}}(\tau)$ provides a direct link between the original and mapped variables and assesses the accuracy of the mapping. In particular, deviations between the numerically computed mapped energy and its expected analytical behaviour are used to quantify the error introduced by the numerical scheme. Although the original energy E is conserved, the mapped energy $E_{\text{map}}(\tau)$ is not constant in time due to the rescaling introduced by the transformation, and instead follows a known analytical evolution.

4.2 Mapping the Inviscid Burgers' Equation

To implement the mapping described in Bustamante 2011 p. 10 for the inviscid Burgers' equation, the mapped time is defined by

$$\tau(t) = -\ln(t_s - t). \tag{4.4}$$

This expression follows from the definition of the mapped time variable (4.1) together with equation (3.9), where the shock time $t_s = 1$. Under this transformation, the inviscid Burgers' equation becomes

$$v_\tau + v\, v_x = -v. \tag{4.5}$$

Introducing an additional damping term $-v$, which arises from the time rescaling. In the numerical implementation, the initial mapped velocity is constructed as

$$v(x, 0) = \frac{f(x)}{\|u_x(\cdot, 0)\|_\infty}. \tag{4.6}$$

The supremum norm $\|u_x\|_\infty$ is not taken directly from grid values. Instead, as before it is found using quadratic interpolation. This interpolation procedure yields a more accurate estimate of the true maximum of $|u_x|$.

After computing each time step, the velocity field is then renormalised according to

$$v(x, \tau) \longleftarrow \frac{v(x, \tau)}{\|v_x(\cdot, \tau)\|_\infty}, \qquad \text{so that} \qquad \|v_x(\cdot, \tau)\|_\infty = 1. \tag{4.7}$$

The mapping is bijective, so there is a one-to-one correspondence between $u(t)$ and $v(\tau)$, meaning the original solution can be uniquely recovered from the mapped solution by inverting the transformation.

4.3 Numerical Results

Similarly to Bustamante (2011), the initial conditions are

$$f(x) = 1 + \cos(x), \quad 0 \le x \le 2\pi, \quad 0 \le t < t_s. \tag{4.8}$$

The resolutions used were $N = (1024, 2048, 4096, 8192)$. These were used so an easy comparison can be shown with Bustamante (2011) since the same resolutions were used (with the exception of 8192, which is used instead of 16384 due to limits on memory).

To determine the accuracy of the method, two errors are used, defined as

$$e_1(\tau) = 2\Big(\tau - \ln\big\|\partial_x u\big(\cdot, t(\tau)\big)\big\|_\infty\Big), \tag{4.9}$$

$$e_2(\tau) = 2\tau + \ln E_{\text{map}}(\tau) - \ln\big(1.5\,\pi\big). \tag{4.10}$$

The errors are designed to measure deviations from the known asymptotic behaviour of the solution near singularity formation. In the mapped variables, both the gradient and the energy change in a predictable way that is known in advance. These errors compare the numerical solution to these scalings, and should remain constant if the method is accurate. Specifically, e_1 tests the accuracy of the gradient growth, while e_2 measures how well the mapped energy follows its expected evolution. Deviations therefore indicate numerical error, particularly close to the singularity.

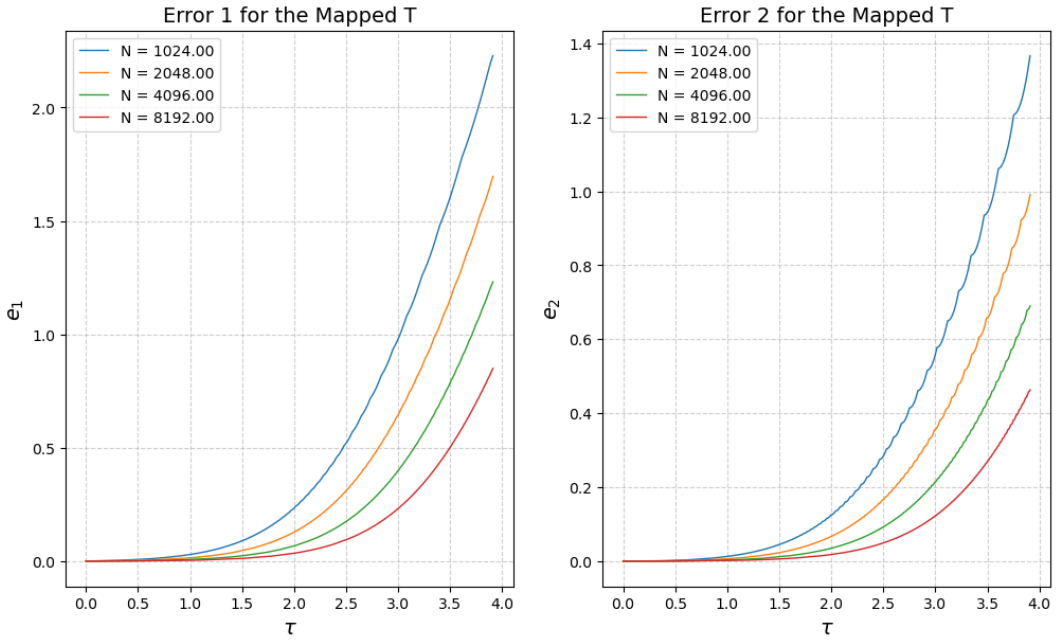


Figure 4.1: *Error analysis from numerically finding the solution to the mapped inviscid Burgers' equation using the time mapping scheme and first-order upwind method. They can be seen to diverge quickly as τ increases.* ***Left panel:*** *The e_1 error.* ***Right panel:*** *The e_2 error.*

The errors found, shown in figure 4.1, clearly diverge far too early compared to the results found in Bustamante (2011). The term divergence (of the errors) is meant here to say when the absolute value of the error starts to increases without bound. Both errors do however follow predictable behaviour as they decrease as the resolution increases. To improve these results, a stronger and higher-order

method is needed, a spectral method is used in Bustamante (2011). This method is much more accurate and can stay reliable closer to the singularity.

5 Spectral Method with Fourier Basis

5.1 Defining the Spectral Method

A new method is now used to increase the accuracy and decrease the errors of the previous results. Still using the inviscid Burgers' equation and periodic boundary conditions (a requirement for the spectral method) in equation (3.1).

The first step in the process is finding an approximate representation of $u(x,t)$ using a truncated Fourier series

$$u(x,t) \approx \sum_{k=-N/2}^{N/2-1} \hat{u}_k(t) e^{ik\kappa x}, \quad \kappa = \frac{2\pi}{L}. \tag{5.1}$$

Now the spatial derivative of $u(x,t)$ can easily be found in Fourier space

$$u_x \approx \sum_{k=-N/2}^{N/2-1} ik\kappa \hat{u}_k(t) e^{ik\kappa x}.$$

Here L means the length of the region of x and in this case $L = 2\pi$ (equation (3.7)) is being used, so $\kappa = 1$. The non-linear term cannot be evaluated directly in Fourier space because multiplication in physical space corresponds to a convolution in Fourier space, which couples all modes and leads to an $\mathcal{O}(N^2)$ computational cost. Instead, a pseudo-spectral approach is used, where the product is computed in physical space and transformed back efficiently using FFTs (Fast Fourier Transforms) represented as $\mathcal{F}$ and $\mathcal{F}^{-1}$ for IFFTs (Inverse Fourier Transforms).

$$\hat{u}_k \approx \mathcal{F}(u), \quad u \approx \mathcal{F}^{-1}(\hat{u}_k).$$

5.2 Implementing the Method

Implementing the pseudo-spectral method in Python follows a simple step by step process which repeats for each time step:

1. Starting in Fourier space at $t = 0$, transform $\hat{u}_k$ into physical space

$$u = \mathcal{F}^{-1}(\hat{u}_k).$$

2. Compute the spatial derivative spectrally:

$$u_x = \mathcal{F}^{-1}(ik\hat{u}_k).$$

3. Form the non-linear product:

$$g = u \cdot u_x.$$

4. Transform back to Fourier space:

$$\hat{g} = \mathcal{F}(g).$$

5. Then dealiasing needs to be applied since the non-linear product generates high-frequency modes that cannot be represented on the discrete grid, leading to aliasing errors. To control this, the 2/3 dealiasing rule is applied:

$$\hat{g} = 0 \quad \text{for} \quad |k| > \frac{k_{max}}{3}.$$

   This ensures that only the resolved modes are retained and improves stability and accuracy.

6. Finally the negative of $\hat{g}$ is taken to compute the time derivative

$$\frac{d\hat{u}}{dt} = -\hat{g}.$$

Time is now advanced using the fourth-order Runge Kutta method (RK4). RK4 is an effective method that achieves fourth-order accuracy in time. Given the solution $\hat{u}_j$ at time t^n, the solution at the next time step $t^{n+1} = t^n + \Delta t$ is computed as

$$\hat{u}_{j+1} = \hat{u}_j + \frac{\Delta t}{6}\left(K_1 + 2K_2 + 2K_3 + K_4\right), \tag{5.2}$$

where the intermediate stages are defined by

$$\begin{aligned}
K_1 &= F(\hat{u}_j), \\
K_2 &= F\left(\hat{u}_j + \frac{\Delta t}{2}K_1\right), \\
K_3 &= F\left(\hat{u}_j + \frac{\Delta t}{2}K_2\right), \\
K_4 &= F\left(\hat{u}_j + \Delta t\, K_3\right).
\end{aligned}$$

Here $F(\hat{u}_k)$ represents the non-linear term. The time step Δt is still chosen according to a CFL condition. Lastly the solution in physical space is obtained via the inverse FFT for each time step

$$u(x,t) = \mathcal{F}^{-1}(\hat{u}_k).$$

Second-order interpolation is still used to calculate the infinity norms.

This pseudo-spectral method is also used on the time mapping used before. The Fourier transform procedure is the same apart from the fourth step which has the extra term, turning it into

$$\hat{g} = \mathcal{F}(g) + \hat{v}_k.$$

5.3 Numerical Results

The first test used the same initial conditions as Section 3.3, allowing direct comparison with the upwind–forward Euler scheme. As in Chapter 3, nineteen uniform resolutions were taken, staring at 100 and ending at 1000.

$$f(x) = \sin(x), \quad 0 \le x \le 2\pi, \quad 0 \le t < t_s.$$

In figure 5.1 the left panel recreates figure 3.2's left panel. It is hard to tell based of the image which method is more accurate, however, as can be seen in the right panel, the errors are now drastically lower than before (shown in figure 3.3's right panel).

An analysis was then done to determine the usefulness of the dealiasing. The same experiment was tested as in figure 5.1 but without dealiasing. Shown in figure 5.2, the dealiasing errors were subtracted from the non-dealiased errors at $t = 0.5$ for each resolution.

The figure shows that the inclusion of dealiasing did not significantly alter the error. This is possibly because the solution does not develop sharp features until late in the simulation, so aliasing errors remain small. The dominant errors could be arising from time integration rather than spectral aliasing, lessening dealiasing usefulness at this time ($t = 0.5$).

Lastly the time-mapping was then tested, now using the spectral method. Using the same conditions as Section 4.3

$$f(x) = 1 + \cos(x), \quad 0 \le x \le 2\pi, \quad 0 \le t < t_s,$$

and using the same errors e_1 and e_2 ((4.9) and (4.10) respectfully) with $N = (1024, 2048, 4096, 8192)$ as the resolutions. Powers of 2 are used for the resolution

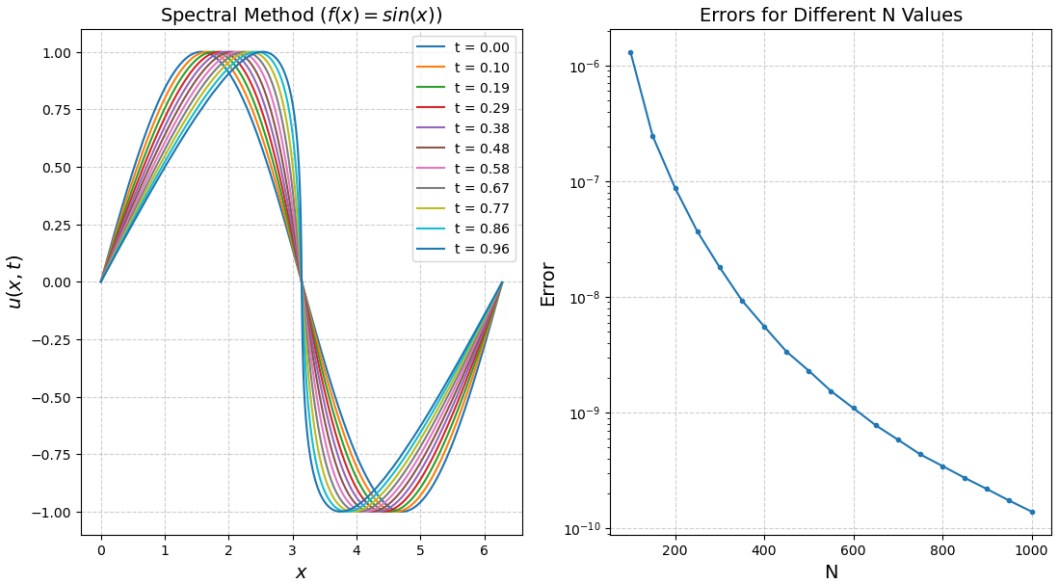


Figure 5.1: *Using the spectral method on the initial conditions* $f(x) = \sin(x)$. ***Left panel:*** *Plotting* $u(x, t)$ *for* $N = 1000$. ***Right panel:*** *The error at* $t = 0.5$ *for each resolution.*

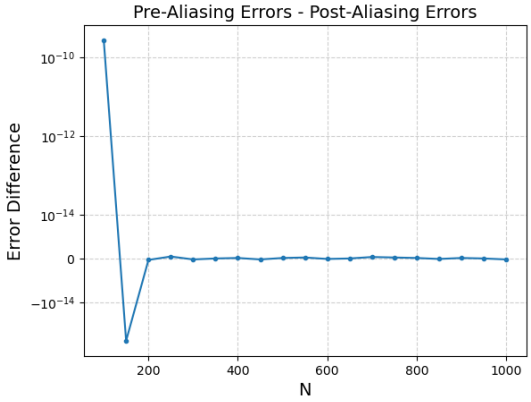


Figure 5.2: *The plotted difference in errors between using dealiasing in the spectral method and without using it. Dealiasing can be seen to have very little effect for* $N \geq 200$.

because the fast Fourier transform (FFT) algorithm is most efficient when the number of grid points is $N = 2^k$, allowing the transform to be computed using repeated halving. This minimises computational cost and is standard practice in Fourier spectral methods.

Clearly figure 5.3 shows a massive improvement compared to earlier. Divergence of the errors occurs at around $\tau = 1$ for e_1 and e_2 while in figure 5.3, divergence occurs at around $\tau = 2.5$ and $\tau = 3$ for e_1 and e_2 respectfully (depending on the resolution). The errors still diverge later for higher resolutions as expected but now they all diverge far later than previously.

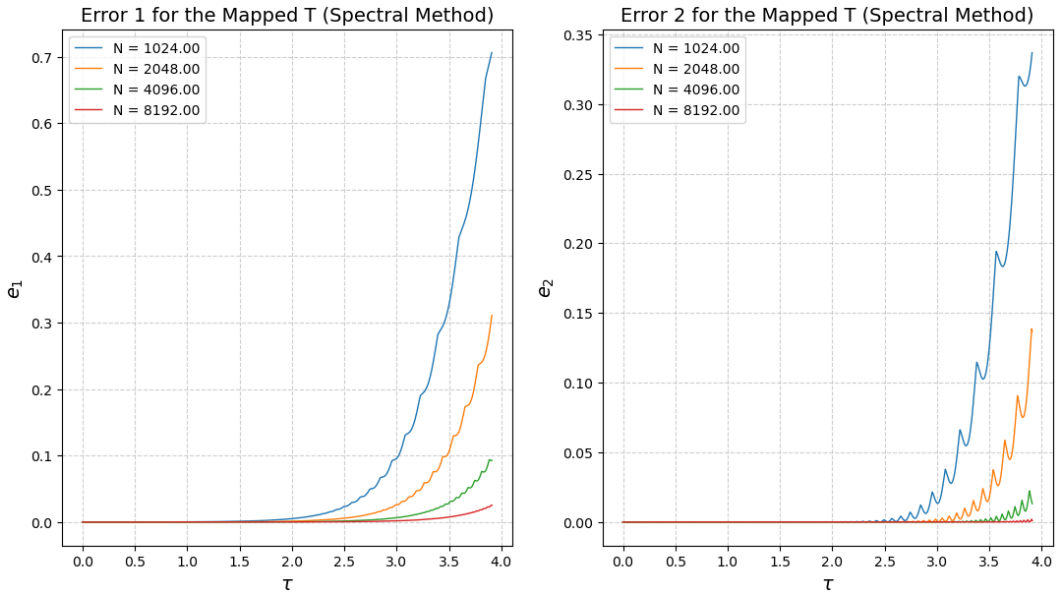


Figure 5.3: *Error analysis from the mapped inviscid Burgers' equation using the spectral method with Fourier basis. Noticeable improvement can clearly be seen, the errors diverge later than figure 4.1 and results still get better with higher resolutions.* ***Left panel:*** *The e_1 error.* ***Right panel:*** *The e_2 error.*

Figure 5.4 was made by taking the data from the resolution of 8192 (the most accurate data) and slicing 50 evenly spaced points along τ. These points were then plotted on the same axis as the results for e_2 from Bustamante (2011).

The results now resemble the data found by Bustamante (2011). Very small errors are observed up until the they diverge around $\tau = 2.7$. This is earlier than shown in the paper where the best resolution data can be seen to diverge around $\tau = 3.9$. However, as mentioned before, their highest resolution is 16384 and they use order-16 interpolation to find the infinity norm. This is most likely the reason for the differing results, based on figure 5.3 it would follow that a higher resolution would show a line flatter than even the red line (resolution $N = 8192$) at $\tau \approx 4$.

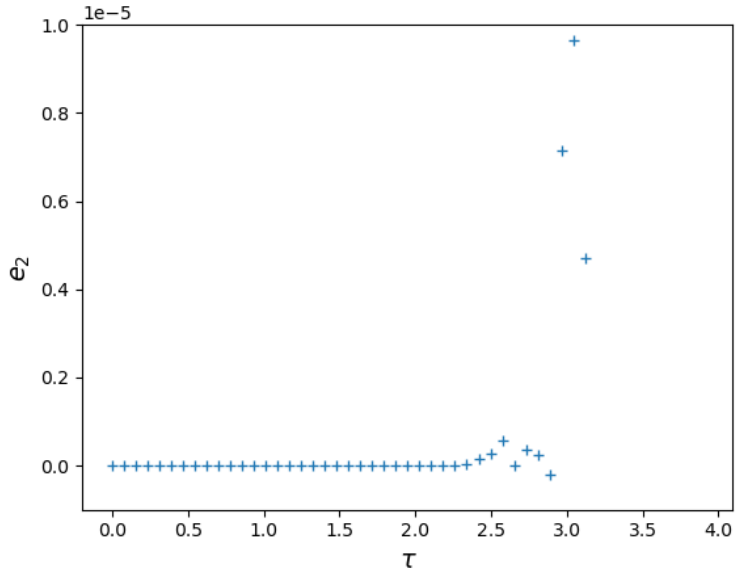


Figure 5.4: *Error analysis of e_2 from figure 5.3 using the $N = 8192$ resolution and using similar axis to Bustamante (2011).*

6 Conclusion

This project investigated the formation of singularities in partial differential equations using the example of the inviscid Burgers' equation. Beginning with the linear advection equation provided a simple test case where analytical solutions are known and no singularities form. This allowed the numerical methods to be validated before moving to the non-linear case.

The inviscid Burgers' equation was then studied, where the dependence of the velocity on the solution leads to the formation of shocks in finite time. Numerical solutions using a first-order upwind scheme were able to capture the general behaviour of the solution, however the accuracy rapidly deteriorated as the shock time was approached. This was particularly evident in the growth of the gradient, where the numerical method struggled to accurately resolve the blow-up.

To better analyse the behaviour near the singularity, a time-mapping technique was implemented. This successfully reformulated the problem so that the singularity occurs at infinite mapped time, allowing the solution to be studied closer to the blow-up. However, despite this improvement, the numerical errors still grew quicker than Bustamante (2011), indicating that the underlying numerical method was not sufficiently accurate.

To address this, a spectral method using a Fourier basis was introduced. This provided a significant improvement in accuracy, as spatial derivatives could be computed with much higher precision. The results showed that the errors e_1 and e_2 remained small for much longer times, and their divergence was delayed compared to the finite-difference approach. Increasing the resolution further improved the results, confirming the expected convergence of the method.

Although the results obtained using the spectral method closely resemble those found in Bustamante (2011), there are still some differences. In particular, the divergence of the errors occurs earlier than expected. This is likely due to limitations in resolution and the use of lower-order interpolation when estimating the infinity norm. Despite this, the overall behaviour is consistent, and the method provides a much more accurate representation of the solution near singularity formation.

Overall, this project demonstrates that higher-order numerical methods are essential for accurately studying singularities in non-linear PDEs. The spectral method, combined with the time-mapping approach, provides a strong system

for investigating behaviour close to blow-up, and offers clear advantages over standard low-order schemes.

To extend this work, it would be natural to consider alternative numerical methods that are better suited to resolving the sharp gradients that develop near the singularity. While the spectral method provides high accuracy, it is known that such methods can lose their effectiveness when the solution develops singularities unless sufficient resolution is used. As shown by Basdevant et al. (1986), this issue can be improved by concentrating grid points in regions of large gradients, either through coordinate transformations or by using spectral element methods. Incorporating these ideas into the current framework would likely improve the accuracy near blow-up, particularly by delaying the growth of the errors and allowing the solution to be resolved closer to the singularity.

Another direction would be to improve the time integration scheme. In this project, relatively low-order time-stepping methods were used, which may contribute to the early growth of numerical error. However, as demonstrated in Seydaoğlu, Erdoğan, and Öziş (2016), higher-order splitting and extrapolation methods can significantly improve accuracy for Burgers' equation, especially in regimes with small viscosity. In particular, sixth-order extrapolation methods provide much more accurate results than standard second-order schemes. Applying these higher-order methods in combination with the spectral discretisation and mapping approach would provide a more accurate framework for studying the behaviour near singularity formation. More generally, extending these techniques to more complex equations, such as the Euler equations, would be a natural next step in further investigating singularities in non-linear PDEs.

Bibliography

Basdevant, C. et al. (1986). "Spectral and finite difference solutions of Burgers equation". In: *Computers Fluids* 14, pp. 23–41. DOI: 10.1016/0045-7930(86)90036-8.

Burgers, J. M. (1948). "A mathematical model illustrating the theory of turbulence". In: *Advances in Applied Mechanics* 1, pp. 171–199. URL: https://api.semanticscholar.org/CorpusID:117400324.

Bustamante, M. D. (2011). "3D Euler equations and ideal MHD mapped to regular systems: Probing the finite-time blowup hypothesis". In: *Physica D: Nonlinear Phenomena* 240.13, pp. 1092–1099. DOI: https://doi.org/10.1016/j.physd.2011.03.006.

Courant, R., K. Friedrichs, and H. Lewy (1928). "Über die partiellen Differenzengleichungen der mathematischen Physik". In: *Mathematische Annalen* 100.1, pp. 32–74.

Fefferman, C. L. (2000). *Existence and Smoothness of the Navier–Stokes Equation*. URL: https://www.claymath.org/millennium/navier-stokes-equation/ (visited on 04/24/2026).

Jost, J. (2002). *Partial Differential Equations*. Springer.

Lucas, D. (2025). *MT4112 Computational Numerical Analysis Lecture Notes*. University of St Andrews.

Seydaoğlu, M., U. Erdoğan, and T. Öziş (2016). "Numerical solution of Burgers' equation with high order splitting methods". In: *Journal of Computational and Applied Mathematics* 291, pp. 410–421. DOI: https://doi.org/10.1016/j.cam.2015.04.021.

Tadmor, E. (1989). "Convergence of Spectral Methods for Nonlinear Conservation Laws". In: *SIAM Journal on Numerical Analysis* 26.1, pp. 30–44. URL: http://www.jstor.org/stable/2157705.

Tao, T. (2007). *Why global regularity for Navier-Stokes is hard*. URL: https://terrytao.wordpress.com/2007/03/18/why-global-regularity-for-navier-stokes-is-hard/ (visited on 04/24/2026).

Trefethen, L. N. (2000). *Spectral Methods in MATLAB*. SIAM (Society for Industrial and Applied Mathematics).

A | Python Code for the Numerical Methods Used

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML
import sympy as sp
from scipy.optimize import curve_fit
from scipy.optimize import fsolve
from scipy import integrate
```

Listing A.1: *Libraries used throughout project*

```
### Linear Advection using Upwind Euler ###

def Euler(u, f, t, x):

   delta_x = x[1] - x[0]
   delta_t = t[1] - t[0]
   delta_xt = delta_t/delta_x

   # Copy initial condition
   u = u.copy()
   u_big = np.zeros((len(t), len(x)))
   u_big[0] = u.copy()

   # Time stepping loop
   for i in range(1, len(t)):
      u_new = u.copy()
      v_vals = f(t[i], x) # velocity at current time/space
      pos_mask = v_vals > 0 # upwind direction depends on
         velocity sign

      # backward diff (for v>0)
```

```
        backward = (u - np.roll(u, 1))

        # forward diff (for v<0)
        forward = (np.roll(u, -1) - u)

        # choose correct derivative for each point
        u_x = np.where(pos_mask, backward, forward)

        # update all points at once
        u_new = u - delta_xt * v_vals * u_x
        u = u_new

        # store solution
        u_big[i] = u.copy()
    return u_big
```

Listing A.2: *Linear Advection Numerical Implementation*

```
def Euler_Burgers(u, t, x):

    delta_x = x[1] - x[0]
    delta_t = np.diff(t)
    delta_xt = delta_t/delta_x

    # Copy initial condition
    u = u.copy()
    u_big = np.zeros((len(t), len(x)))
    u_big[0] = u.copy()

    # Time stepping loop
    for i in range(1, len(t)):
        v_vals = u # characteristic speed = u
        pos_mask = v_vals > 0 # upwind direction depends on
            velocity sign

        # backward diff (for v>0)
        backward = (u - np.roll(u, 1))

        # forward diff (for v<0)
        forward = (np.roll(u, -1) - u)

        # choose correct derivative for each point
        u_x = np.where(pos_mask, backward, forward)
```

```
        # update all points at once
        u = u - delta_xt[i-1] * v_vals * u_x

        # store solution
        u_big[i] = u.copy()

    return u_big
```

Listing A.3: *Inviscid Burgers' Equation Numerical Implementation*

```
# Shock Time

def ShockTime_Burgers(x, u0):
    dx = x[1] - x[0]
    du_dx = np.gradient(u0, dx) # Finds gradient
    ShockTime = -1/(min(du_dx))
    return ShockTime
```

Listing A.4: *Numerically Finding the Shock Time*

```
# Mapped Time

def Mapped_Euler_Burgers(u, tau, x):
    dx = x[1] - x[0]
    dtau = np.diff(tau)

    ux_0 = (np.roll(u, -1) - np.roll(u, 1)) / (2 * dx)
    ux_inf = np.max(np.abs(ux_0))
    v0 = u / ux_inf

    # initial mapped field
    v = v0.copy()

    # storage
    v_big = np.zeros((len(tau), len(x)))
    v_big[0] = v.copy()

    for i in range(1, len(tau)):
        v_vals = v
        pos_mask = v_vals > 0 # upwind direction depends on
            velocity sign

        # upwind derivatives
```

```
        backward = (v - np.roll(v, 1)) / dx
        forward = (np.roll(v, -1) - v) / dx
        # choose correct derivative for each point
        v_x = np.where(pos_mask, backward, forward)

        v = v - dtau[i-1] * (v_vals * v_x + v)

        # normalisation solution
        v_x_full = (np.roll(v, -1) - np.roll(v, 1)) / (2 * dx)
        norm = np.max(np.abs(v_x_full))
        v /= norm

        # store solution
        v_big[i] = v.copy()

    return v_big
```

Listing A.5: *Inviscid Burgers' Equation Numerical Implementation Using the Time Mapping*

```
def ThreePointInterpolation(x):
    grid = len(x)
    if x.ndim == 2:
        idx = np.argmax(np.abs(x), axis = 1)
        t_idx = np.arange(len(idx))
    else:
        idx = np.argmax(np.abs(x))

    i0 = (idx - 1) % grid
    i1 = idx
    i2 = (idx + 1) % grid
    if x.ndim == 2:
        y0 = np.abs(x[t_idx, i0])
        y1 = np.abs(x[t_idx, i1])
        y2 = np.abs(x[t_idx, i2])
    else:
        y0 = np.abs(x[i0])
        y1 = np.abs(x[i1])
        y2 = np.abs(x[i2])

    return y1 - (y2 - y0)**2 / (8*(y2 - 2*y1 + y0))
```

Listing A.6: *Second-Order Interpolation*

```
# Pseudo Spectral Method using Fourier Basis

def SpectralMethodInviscidBurgers(u0, L, x, t):
   N = len(x)
   dt = np.diff(t)

   uhat = np.fft.fft(u0)
   k = 2*np.pi*np.fft.fftfreq(N, d=L/N)

   u_big = np.zeros((len(t), len(x)))
   u_big[0] = u0

   def f(uhat, k):
      u = np.fft.ifft(uhat)
      ux = np.fft.ifft(1j*k*uhat)
      f = u * ux
      fhat = np.fft.fft(f)
      fhat[np.abs(k) > N / 3] = 0.0
      duhat = -fhat
      return duhat
   for i in range(1, len(t)):
      K1 = f(uhat, k)
      K2 = f(uhat + 0.5 * dt[i-1] * K1, k)
      K3 = f(uhat + 0.5 * dt[i-1] * K2, k)
      K4 = f(uhat + dt[i-1] * K3, k)
      uhat = uhat+(K1 + 2*K2 + 2*K3 + K4)*dt[i-1]/6
      u_big[i] = np.real(np.fft.ifft(uhat))
   return u_big
```

Listing A.7: *Spectral Method Numerical Implementation*

```
def Mapped_SpectralMethodInviscidBurgers(u0, L, x, t):
   dx = x[1] - x[0]
   N = len(x)
   dt = np.diff(t)

   k = 2*np.pi*np.fft.fftfreq(N, d=L/N)
   uhat0 = np.fft.fft(u0)
   ux0 = np.real(np.fft.ifft(1j * k * uhat0))

   ux_inf = np.max(np.abs(ux0))
   v0 = u0 / ux_inf

   uhat = np.fft.fft(v0)
```

```
    k = 2*np.pi*np.fft.fftfreq(N, d=L/N)
    u_big = np.zeros((len(t), len(x)))
    u_big[0] = v0

    def f(uhat, k):
        u = np.fft.ifft(uhat)
        ux = np.fft.ifft(1j*k*uhat)
        f = u * ux
        fhat = np.fft.fft(f) + uhat
        fhat[np.abs(k) > N / 3] = 0.0
        duhat = -fhat

        return duhat

    for i in range(1, len(t)):
        K1 = f(uhat, k)
        K2 = f(uhat + 0.5 * dt[i-1] * K1, k)
        K3 = f(uhat + 0.5 * dt[i-1] * K2, k)
        K4 = f(uhat + dt[i-1] * K3, k)
        uhat = uhat+(K1 + 2*K2 + 2*K3 + K4)*dt[i-1]/6

        # normalisation solution
        v = np.real(np.fft.ifft(uhat))
        vx = np.real(np.fft.ifft(1j * k * uhat))

        norm = ThreePointInterpolation(vx)
        uhat /= norm

        u_big[i] = np.real(np.fft.ifft(uhat))
    return u_big
```

Listing A.8: *Spectral Method Numerical Implementation Using the Time Mapping*